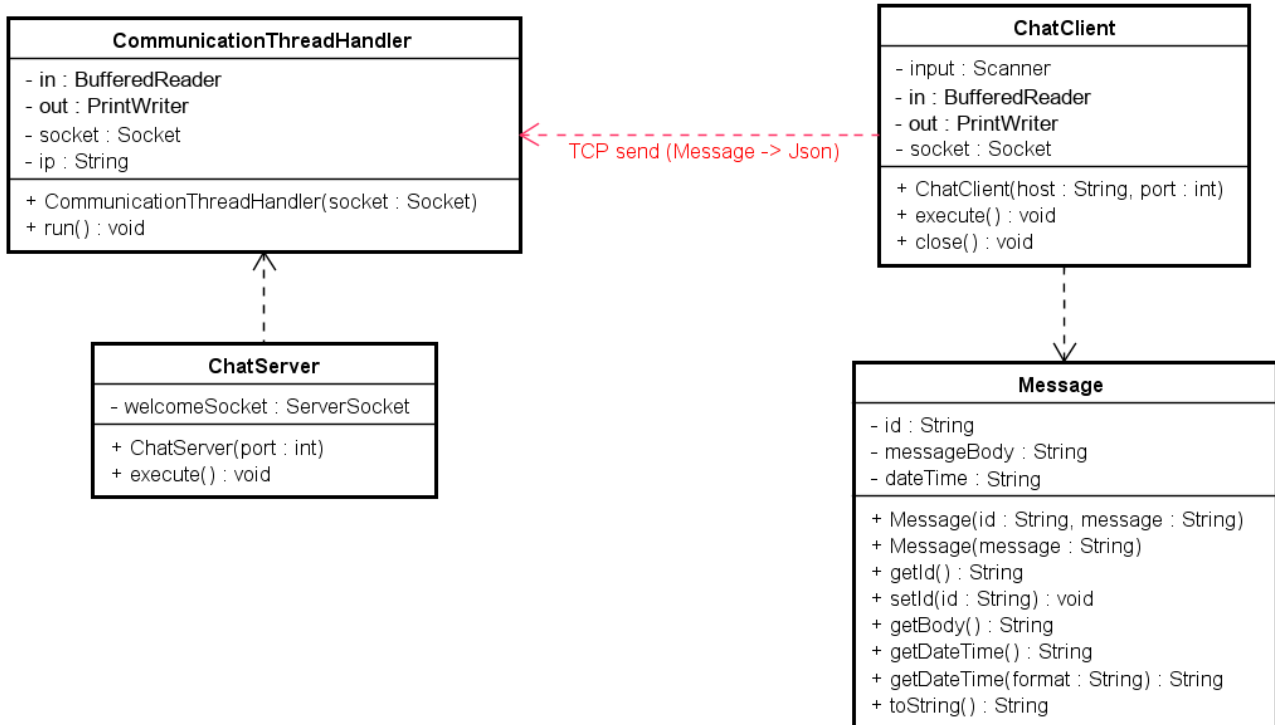


Exercise: A “chat” where clients send messages to the server



Step 1: Implement the Model

Implement class `Message` – or copy from Appendix A last in this document

Step 2: Implement the Server side

Step 2A: Implement the Server side (Thread handler)

Implement class `CommunicationThreadHandler`.

- implementing `Runnable`
- The constructor is initializing instance variables
- Method `run` with a loop reading a `Json` string from the client, converting this to a `Message` object, and simply printing out the object. End the loop if the body of the message is “EXIT”.

Step 2B: Implement the Server side (ChatServer)

Implement class `ChatServer`.

- The constructor is initializing instance variables
- Method `execute` creates an infinite loop in which a client socket is created (`ServerSocket` method `accept()`) and a thread (with a `CommunicationThreadHandler` object) is created and started.

Step 2C: Implement the Server side (Server main)

Implement class `Server` with a main method, creating a `ChatServer` and calling `execute`.

Step 3: Implement the Client side

Step 3A: Implement the Client side (TaskListClient)

Implement class `ChatClient`.

- a) The constructor is initializing instance variables
- b) Method `execute` creates a loop in which you repeatedly
 - 1) Read an input text from keyboard
 - 2) Create a `Message` object with the input text as the message body
 - 3) Convert the `Message` object to a `Json` string
 - 4) Send the `Json` string to the server
- c) Method `close` closes the socket and the keyboard stream (`Scanner` object)

Step 3B: Implement the Client side (Client main)

Implement class `Client` with a main method, creating a `ChatClient` and calling `execute`.

Appendix A: Class Message

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Message
{
    private String id;
    private String messageBody;
    private String dateTime;
    public Message(String id, String message)
    {
        this.dateTime =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd/MM/yy hh:mm:ss"));
        this.id = id;
        this.messageBody = message;
    }
    public Message(String message)
    {
        this("0", message);
        setId("" + (int) (messageBody.hashCode() * Math.random()));
    }
    public Message update()
    {
        this.dateTime =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd/MM/yy hh:mm:ss"));
        return this;
    }
    public String getId()
    {
        return id;
    }
    public void setId(String id)
    {
        this.id = id;
    }
    public String getBody()
    {
        return messageBody;
    }
    public String getDateTime()
    {
        return dateTime;
    }
    public String toString()
    {
        DateTimeFormatter formatter
            = DateTimeFormatter.ofPattern("d/MM/yyyy HH:mm:ss");
        return "id=" + id + ", time=" + dateTime
            + ", message=\"" + messageBody + "\"";
    }
}
```